

AI 使用说明情况（第三次挑战）

学号 0102603133

一、使用的 AI 模型

工具：CodeBuddy（代理式编码助手），由对话式大模型驱动。使用方式是在沙箱环境里以「自主执行」模式工作：它自己规划步骤、通过 SSH 在 AutoDL 实例（4 × RTX 4090 D）上执行命令、失败后自行重试并向我汇报；我负责目标设定、关键决策（如是否更换基础镜像）和结果验收。全程未使用其它 AI 工具。

二、使用的 AI 提示词

初始提示词（大意）：附上三份挑战 PDF、HW3-scaffold.zip 和远程实例的 SSH 信息，要求从零开始完成第三次挑战：部署 4 个 SGLang 0.5.14 后端（Qwen3-0.6B）、按 1 head + 4 worker 建立 Ray 2.56.0 逻辑集群、用课程固定负载跑完 A/B1/B2/C/D 五轮对照，并按交付格式整理 README、report、src 与 results。**过程中的追加指令：**「需要更换镜像吗」「版本不是太低了吗」「换个高一点的」「我用的是 autodl 可以在控制台直接更换 torch 的版本」「？」——分别触发了 pip 源测速选型、torch 从 2.8.0 升级到 2.11.0、安装方案调整和进度确认。其余时间以简短确认（如 ok）推进。

三、AI 如何帮助你学习

环境工程：学会了用实测而不是口碑来选 pip 源（阿里云 0.17–0.6 MB/s，清华约 16 MB/s）；用两个独立 venv 隔离 SGLang 与 Ray 的依赖、只让它们通过 HTTP 通信；用 hf-mirror 绕开 huggingface.co 的网络不可达问题。**源码级理解：**在 AI 带领下翻了 Ray 2.56 的请求路由源码，搞清楚了内置路由器的继承链（FIFOMixin、LocalityMixin、MultiplexMixin、RequestRouter）、choose_replicas 的 rank 语义（多 rank = 顺序重试而非并行探测），以及 SGLang /flush_cache 的实际响应格式。**指标理解：**把「prefill 削减 89% 但 TPOT 只小幅变化」与 TTFT/TPOT 的定义关联起来，理解了前缀缓存只作用于 prefill 路径。**调试方法论：**组 D 的故障定位示范了一条排查路径——先看失败样本的异常类型，再分代理侧/副本侧看日志，最后对照内置实现找自定义类缺了什么。

四、在运用 AI 时是否被误导

有，而且不止一次，按时间顺序如实列出：（1）**升级建议的陷阱：**AI 最初建议「更换 AutoDL 基础镜像来升级 torch」，那会清空系统盘、重装全部环境；我提出控制台可以直接换镜像后，AI 权衡利弊改为在现有环境内 pip 升级 torch 2.11.0+cu130，保住了已装好的 CUDA 运行库。（2）**protobuf 版本：**AI 先降到 4.25.8，引发依赖冲突；绕了一圈才确认 6.33.5 才是与 Ray 2.56 兼容的正确版本（7.x 会报 FieldDescriptor 无 label 属性）。（3）**组 D 路由器（最大的弯路）：**AI 写的自定义路由器最初缺少 FIFOMixin，首跑 405/2048 请求失败、吞吐仅 5 req/s；它先修了 rank 语义和同步上报两个表层问题，最后对照 Ray 内置 p2c 的继承链才找到根因，补上后恢复到 62.99 req/s。（4）**API 假设过时：**Ray 2.56 的 serve.run() 已删除 host/port 参数、SGLang 0.5.14 的 /flush_cache 返回纯文本而非 JSON，AI 的第一版代码都按旧假设写，运行时报错后才修正。**教训：**AI 给出的 API 细节必须以本机安装版本的真实源码/报错为准，尤其是小版本迭代很快的生态。

五、边界声明

所有实验数字（吞吐、缓存命中率、实际 prefill tokens、TTFT、延迟、后端分布）均来自课程 run_workload.py 的真实运行产物（results/ 下的原始 CSV/JSON 可逐一对齐），AI 只承担汇总、画图与文字表述；src/ 下全部代码都在实例上实际运行验证过（五轮 2048/2048 成功）。按课程要求，作业感受第（2）（3）问由本人独立撰写，未使用 AI。